

INFORME TÉCNICO

Programación Concurrente, Paralelismo de Tareas y Paralelismo de Datos

Asignatura: Sistemas Operativos

Actividad práctica I corte

Estudiante:

Luis Mario Ramirez Benalcazar

Código: 20231020166

Cristian Salamanca

Código: 20222020207

17 de septiembre de 2026

Resumen

Se implementaron y evaluaron dos actividades de programación concurrente en Python: análisis de un historial de WhatsApp y suma vectorial a gran escala. Para el chat se compararon las estrategias de paralelismo de tareas y de datos. Para la sumatoria se contrastó un recorrido monohilo con una implementación multihilo que divide el vector en subrangos continuos. Los resultados muestran que, en las condiciones de prueba, la estrategia por tareas fue ligeramente más rápida para el chat, mientras que la versión secuencial fue más rápida que todas las configuraciones multihilo de la suma vectorial.

Palabras clave: hilos, paralelismo de datos, paralelismo de tareas, speedup, eficiencia, sincronización, overhead.

1. Descripción de la solución

Lenguaje y entorno

Se utilizó Python 3 en Visual Studio Code sobre Windows. Para el análisis del chat se empleó `concurrent.futures.ProcessPoolExecutor` con tres procesos trabajadores, dado que el procesamiento incluye conteos y análisis de texto. En la actividad de sumatoria se utilizó el módulo `threading`, creando hilos con `threading.Thread` y esperando su finalización con `join()`. La medición de tiempo se realizó con `time.perf_counter()`, apropiado para intervalos de corta duración.

Actividad 1 — Historial de chat

Se analizaron 155.749 mensajes de usuario del archivo `ChatFacultad.txt`. Las tareas fueron:

- **T1:** obtener la palabra más frecuente.
- **T2:** identificar la fecha con más mensajes.
- **T3:** determinar el día de la semana con mayor actividad.

La estrategia de tareas asignó una tarea a cada trabajador y dio acceso al 100 % del chat. La estrategia de datos dividió el chat en tres segmentos prácticamente iguales de 51.917, 51.916 y 51.916 mensajes, generó métricas parciales y consolidó los conteos en el proceso principal.

Actividad 2 — Suma vectorial

Se generó un arreglo de 10.000.000 de enteros aleatorios entre 1 y 100. La versión secuencial recorrió el rango completo con un acumulador. La versión multihilo repartió el vector en n segmentos continuos, cada hilo calculó una suma parcial y el hilo principal obtuvo la suma final después de ejecutar `join()`.

2. Resultados cuantitativos

Los resultados de la Actividad 1 fueron consistentes entre ambas estrategias: palabra más frecuente “tiene” con 2.703 ocurrencias; fecha de mayor conversación 30/08/2023 con 4.612 mensajes; y día más activo Jueves con 29.239 mensajes. Esto valida que la reducción de resultados parciales de la estrategia por datos produjo la misma respuesta global que el procesamiento completo por tareas.

Actividad 1

Cuadro 1: Comparación de estrategias para el análisis del chat

Estrategia	Tiempo (s)	Tiempo (ms)	Diferencia frente a tareas
Paralelismo de tareas	4.236994	4,236.994	Referencia
Paralelismo de datos	4.372175	4,372.175	+135.181 ms (+3.19 %)

En la Actividad 1, el paralelismo de tareas fue 135,181 ms más rápido que el paralelismo de datos, una diferencia de 3,19 %. Aunque ambas alternativas producen los mismos resultados, la partición de datos añade trabajo de división, procesamiento local y consolidación de contadores.

Actividad 2

Para la Actividad 2 se tomó como línea base $T_1 = 0.479295$ s, correspondiente a la suma secuencial con un hilo. El tiempo de generación del vector secuencial fue 16.557250 s, pero no se incorporó en las métricas de speedup: se comparó únicamente el tiempo del cálculo de la suma, que es el componente bajo estudio.

Cuadro 2: Resultados de la suma vectorial

n	Tiempo T_n (s)	Tiempo T_n (ms)	Speedup S_n	Eficiencia E_n
1	0.479295	479.295	1.0000	100.00 %
2	0.775812	775.812	0.6178	30.89 %
4	0.743540	743.540	0.6446	16.12 %
8	0.786328	786.328	0.6095	7.62 %
16	0.769907	769.907	0.6225	3.89 %

Observación metodológica

Cada ejecución multihilo generó un vector aleatorio nuevo, por lo que las sumas totales difieren entre pruebas. Esto no invalida la medición de tiempos, pues todas las pruebas usaron $N = 10.000.000$ y el mismo rango de valores aleatorios.

Para una comparación experimental más controlada, conviene generar un único vector con una semilla fija, reutilizarlo para todas las configuraciones y repetir cada experimento varias veces, reportando promedio, mínimo y desviación estándar.

3. Métricas y gráfica

La aceleración o *speedup* se calcula como:

$$S_n = \frac{T_1}{T_n} \quad (1)$$

donde T_1 es el tiempo secuencial y T_n el tiempo con n hilos.

La eficiencia se calcula como:

$$E_n = \frac{S_n}{n} \quad (2)$$

Un speedup superior a 1 indicaría mejora respecto a la versión secuencial; en esta prueba todos los valores fueron inferiores a 1, por lo que el paralelismo con hilos no aceleró la suma vectorial.

Figura 1. Tiempo de ejecución

La siguiente gráfica representa los tiempos obtenidos para la suma vectorial en función del número de hilos.

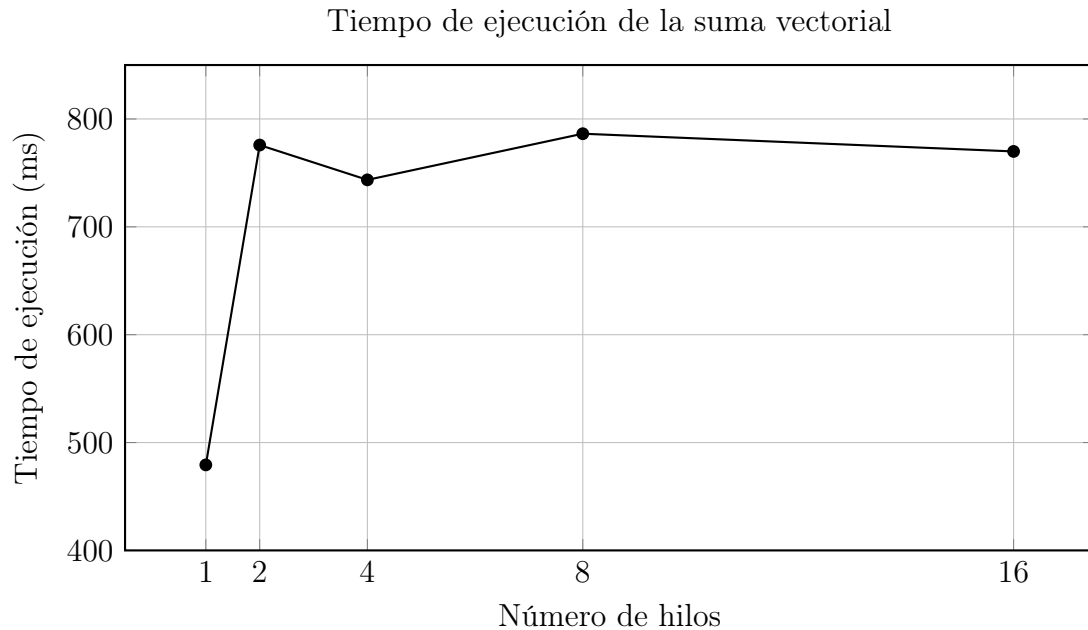


Figura 1: Tiempo de ejecución de la suma vectorial en función del número de hilos.

Interpretación de la gráfica

El menor tiempo fue el de la versión monohilo: 479,295 ms. Entre las versiones multihilo, la mejor configuración fue $n = 4$ con 743,540 ms; aun así, fue 264,245 ms más lenta que la ejecución secuencial.

El punto de saturación práctico se observa antes de $n = 2$: agregar hilos no reduce el tiempo y, a partir de 4 hilos, las variaciones son pequeñas y se mantienen alrededor de 744–786 ms.

4. Discusión técnica

Overhead de hilos

En la suma vectorial hay costos adicionales por crear hilos, asignar rangos, iniciar la ejecución, sincronizar mediante `join()` y efectuar la reducción final de las sumas parciales. Para un único recorrido y una operación muy simple —una suma de enteros— estos costos pueden superar el beneficio esperado de repartir el trabajo.

GIL de Python

La implementación de la Actividad 2 usa `threading` y un ciclo Python de naturaleza intensiva en CPU. En CPython, el Global Interpreter Lock limita la ejecución simultánea de bytecode Python en múltiples hilos. Por ello, los hilos no aprovechan plenamente varios

núcleos para este tipo de carga; el resultado observado es una desaceleración frente a la versión monohilo.

En escenarios de CPU intensiva sería más apropiado usar `multiprocessing`, `ProcessPoolExecutor`, NumPy vectorizado o extensiones compiladas.

Actividad 1

El paralelismo de tareas obtuvo el mejor tiempo (4,236994 s), aunque la diferencia frente al paralelismo de datos fue moderada. La razón es que cada tarea puede avanzar de manera independiente sobre el historial completo, mientras que la estrategia de datos debe dividir el conjunto y después combinar los contadores de palabras, fechas y días.

Sin embargo, el paralelismo de datos mantiene una ventaja conceptual de escalabilidad cuando se pueden asignar bloques a más unidades de cómputo y reducir resultados parciales.

Ley de Amdahl

La ganancia máxima está limitada por la fracción secuencial del programa: creación de estructuras, coordinación, consolidación, acceso a memoria y sincronización.

Aun si la parte paralela fuese muy rápida, estas secciones no paralelizables restringen el speedup. En la Actividad 2, la fracción de coordinación y el costo del modelo de hilos dominan sobre el trabajo útil de cada hilo, lo que explica eficiencias entre 30,89 % con 2 hilos y 3,89 % con 16 hilos.

5. Conclusiones

1. Las dos estrategias de la Actividad 1 entregaron exactamente las mismas métricas globales para 155.749 mensajes, lo que confirma la corrección de la consolidación de resultados parciales.
2. El paralelismo de tareas fue la alternativa más rápida para el análisis del chat: 4,236994 s frente a 4,372175 s para paralelismo de datos, con una diferencia de 3,19 %.
3. En la suma de 10.000.000 de enteros, la versión secuencial fue más eficiente que todas las versiones con `threading`. El mejor resultado multihilo fue $n = 4$, con speedup de 0,6446 y eficiencia de 16,12 %, lo que representa desaceleración en lugar de aceleración.

4. Aumentar el número de hilos no garantiza mejor rendimiento. La elección debe considerar el tamaño y tipo de carga, el costo de sincronización, la memoria y las restricciones del entorno de ejecución.
5. Como mejora experimental, se recomienda usar el mismo vector para todas las pruebas, repetir cada configuración al menos tres veces y emplear procesos o bibliotecas numéricas para cargas intensivas de CPU.

Referencias

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- [2] Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson.
- [3] Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann / Elsevier.
- [4] Guía de actividad práctica I corte: *Programación Concurrente, Paralelismo de Tareas y Paralelismo de Datos*.